COS30019

Introduction to Artificial Intelligence

Assignment 2a: Tree Based Search

Student Name	Student ID
Wei Bao HII	104390572
Stanton Qi Jin CHOO	104391591
Yu Xi PHUAN	104383093
Norman Zhi Wen CHUNG	104387930

TEAM CONTRIBUTION

MEMBERS & ID	TASK	SIGNATURES
Stanton Qi Jin CHOO (104391591) LEADER	● Custom Search Strategy 1 ● Custom Search Strategy 2 ● Research ● Features	
Norman Zhi Wen CHUNG (104387930)	● A* Search Strategy ● GBFS Search Strategy ● Testing ● Insights	
Yu Xi PHUAN (104383093)	● BFS search Strategy ● DFS search Strategy ● Testing ● Insight	
Wei Bao HII (104390572)	● Report Making ● Comparison ● Instruction ● Introduction	

Table of Contents

1. Instructions..... 3
2. Introduction..... 3
3. Features/Bugs/Missing..... 4
4. Testing & Insights..... 4
4. Insights.....9
5. Comparison.....12
6. Research..... 12
7. Conclusion..... 12
8. Acknowledgements/Resources.....13
9. References..... 13

1. Instructions

The program is designed to run using the command-line interface (CLI) in a Windows environment. It supports multiple search algorithms and test cases stored in text files.

Running a Single Test Case :

For carrying out a particular test case, the following command should be used: python search.py <filename> <method>, where filename refers to the name of the test file (such as TC02) while method represents one of the six available search algorithms like DFS, BFS, GBFS, AS (A*), CUS1 (Iterative Deepening Search), and CUS2 (Least Moves Heuristic Search):

```
python search.py PathFinder-test bfs
```

```
Test_Cases/PathFinder-test.txt BFS
4 6
2 -> 1 -> 4
```

To save time from the repetitive process of executing such commands manually, a program known as python run_all_tests.py has been written. The role of this script is to run all test cases using the six available methods. Thus, an output is produced in the form of a table containing a comparison between the performances of various algorithms:

```
================================================================================
TEST CASE: PathFinder-test.txt
================================================================================
Method     Goal       Nodes      Path
--------------------------------------------------------------------------------
DFS        5          7          2 -> 1 -> 3 -> 5
BFS        4          6          2 -> 1 -> 4
GBFS       5          6          2 -> 3 -> 5
AS         4          6          2 -> 1 -> 4
CUS1       4          6          2 -> 1 -> 4
CUS2       5          5          2 -> 3 -> 5
```

The run_all_tests.py file simplifies performance evaluation since it runs all tests at once and provides results in the form of an organized tabular format. The information provided in the tabular format includes the method used to search, the goal state achieved by the algorithm, the number of nodes expanded in the process, and the path that was generated between the start and finish states.

2. Introduction

Route Finding Problem is an important AI problem which involves discovering the most optimal route through a graph to get from starting point(s) to one or more end points(s). In our attempt to solve this problem, our group developed six tree-based algorithm implementations such as DFS, BFS, GBFS, and A*(A Star). In addition, we made use of IDS(CUS1), which tries to find a balance between memory efficiency and completeness, and the Heuristic Least-Moves Search (CUS2).

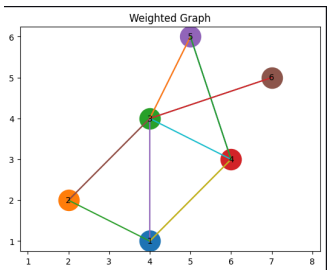
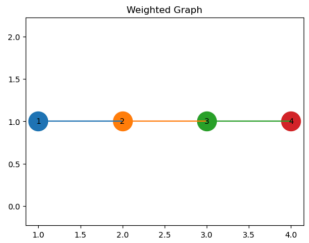
This was done using complicated graphs with directed connections, cycles, and multiple end goals. This paper highlights some of the challenges faced when developing these solutions while considering environmental variables as well.

3. Features/Bugs/Missing

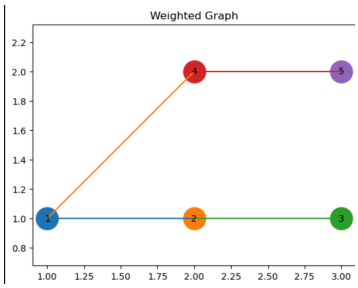
The program implemented all major assignment requirements in the main search.py file. It supports six algorithms: DFS, BFS, GBFS, A*, CUS1 (Iterative Deepening Search), and CUS2 (Heuristic Least-Moves Search). The system can read the required graph text files, handle directed edges, multiple goals, and apply tie-breaking rules. A separate run_all_tests.py file was also created to automatically run all fifteen test cases and display the results in table format. No major bugs were found during final testing. Minor differences in node creation counts may occur, especially for CUS1, because it repeats searches with increasing depth limits. The program also assumes the input file is correctly formatted.

4. Testing & Insights

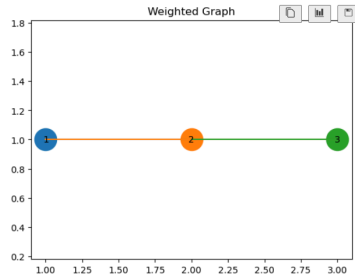
In order to ensure the functionality of the program, fifteen test cases, including the sample provided by PathFinder, were used to test different cases, such as cases with multiple destinations, impossible goal locations (TC05, TC07), cycles, and cases with the starting location as the goal (TC06). All six algorithms used, including DFS, BFS, GBFS, AS, CUS1, and CUS2, were automatically tested against validity of the generated paths and the number of expanded nodes in different graphs. It was proved that the program is perfectly working and reliable as all six algorithms have passed all tests.

No	Graph		Solution					
1		Method	DFS	BFS	GBFS	AS	CUS1	CUS2
		Path	2-1-3-5	2-1-4	2-3-5	2-1-4	2-1-4	2-3-5
		Nodes	7	6	6	6	6	5
		TC01 – Original Pathfinder Sample Graph: Uses the assignment sample graph to verify correctness. Most algorithms reach the goal just by 3 steps except for DFS which did it in 4 steps because it blindly expands nodes and backtracks when it reaches a dead end. CUS2 performed the best here with only 5 nodes created.						
2		Method	DFS	BFS	GBFS	AS	CUS1	CUS2
		Path	1-2-3-4	1-2-3-4	1-2-3-4	1-2-3-4	1-2-3-4	1-2-3-4
		Nodes	4	4	4	4	7	4

<table>
<tr><td></td><td>All edges have cost of 1</td><td>TC02 – Single Destination Simple Path: Tests a small graph with one destination and one clear valid path.

In this test case, CUS1 (IDS) expanded more nodes since the algorithm itself repeatedly explores nodes with DFS with depth limits. This repetition increases the amount of expanded nodes which makes it inefficient.</td></tr>
<tr><td>3</td><td> 

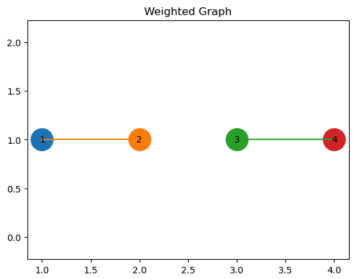
Both 3 and 5 are destinations</td><td><table>
<tr><td>Method</td><td>DFS</td><td>BFS</td><td>GBFS</td><td>AS</td><td>CUS1</td><td>CUS2</td></tr>
<tr><td>Path</td><td>1-2-3</td><td>1-2-3</td><td>1-2-3</td><td>1-2-3</td><td>1-2-3</td><td>1-2-3</td></tr>
<tr><td>Nodes</td><td>4</td><td>5</td><td>4</td><td>4</td><td>5</td><td>5</td></tr>
</table>
TC03 – Multiple Destinations Available: Tests whether the program stops when a reachable goal is found from multiple destinations.

All methods manage to find the shortest path, with the only difference being DFS, GBFS and AS having less expanded nodes. DFS having lower expanded nodes is because the goal happens to be on the first path it explores. For GBFS and AS they prioritize nodes with better heuristics or combined cost values which lead them to the goal faster. BFS and CUS1 both created 5 nodes because they expanded all nodes at depth 1 (nodes 2 and 4) before proceeding deeper, discovering the goal only after exploring both branches at that level. Even though 1-4-5 is also a valid path to the goal, the algorithms expand node 2 first due to node value difference, prioritizing the lower value.</td></tr>
<tr><td>4</td><td> 

1-3 has a cost of 2, 1-2 and 2-3 has a cost of 1 each</td><td><table>
<tr><td>Method</td><td>DFS</td><td>BFS</td><td>GBFS</td><td>AS</td><td>CUS1</td><td>CUS2</td></tr>
<tr><td>Path</td><td>1-2-3</td><td>1-3</td><td>1-3</td><td>1-3</td><td>1-3</td><td>1-3</td></tr>
<tr><td>Nodes</td><td>4</td><td>3</td><td>3</td><td>3</td><td>3</td><td>3</td></tr>
</table>
TC04 – Direct Edge to Goal: Tests whether the algorithm chooses a direct connection from origin to goal.

All methods return the shortest path except DFS because the first path it expands was not the goal node, which leads to it ignoring the direct path to the goal since it was not the first searched path.</td></tr>
</table>

5

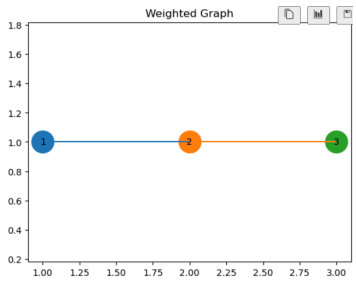


Method	DFS	BFS	GBFS	AS	CUS1	CUS2
Path	-	-	-	-	-	-
Nodes	-	-	-	-	-	-

TC05 – No Reachable Goal: Tests whether the program correctly reports failure when no valid path exists.

All methods did not find a path which is expected because there is no valid path to the goal

6

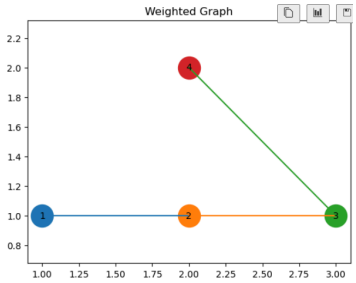


Method	DFS	BFS	GBFS	AS	CUS1	CUS2
Path	1	1	1	1	1	1
Nodes	1	1	1	1	1	1

TC06 – Start Node Is Destination: Tests whether the algorithm immediately returns success when the origin is already a goal node.

All algorithms first check to see if the current node is the goal, in which case it is so all algorithms performed similarly

7

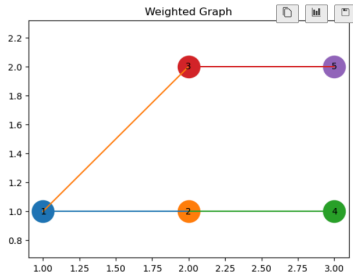


Method	DFS	BFS	GBFS	AS	CUS1	CUS2
Path	-	-	-	-	-	-
Nodes	-	-	-	-	-	-

TC07 – Directed Edge Restriction: Tests whether the program respects one-way edges and prevents illegal reverse movement.

All algorithms respects one-way edges and therefore no valid paths

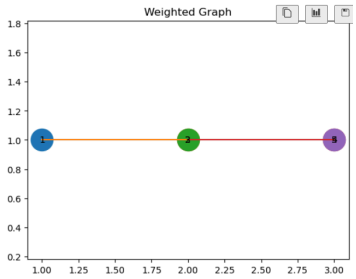
8



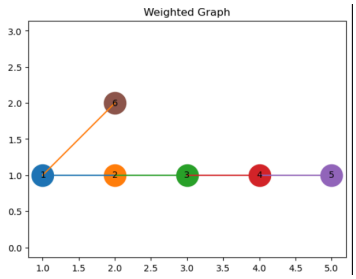
All path costs are equal

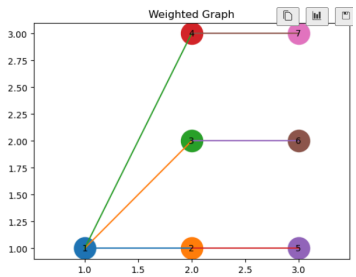
Method	DFS	BFS	GBFS	AS	CUS1	CUS2
Path	1-2-4	1-2-4	1-2-4	1-2-4	1-2-4	1-2-4
Nodes	4	5	4	5	5	5

TC08 – Tie Breaking by Smaller Node Number: Tests whether smaller numbered nodes are expanded first when evaluation values are equal.

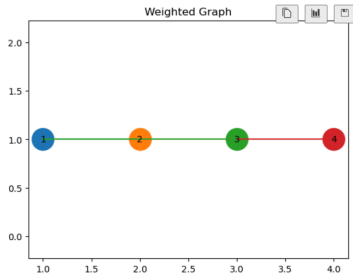
<table>
<tr><td></td><td></td><td>The informed algorithms choose the smaller value with intent while the uninformed algorithms chose the smaller value just because it happened to be the first path it searches</td></tr>
<tr><td>9</td><td> 

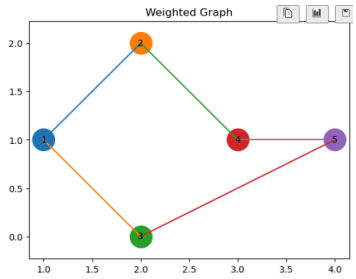
paths 1-2-4 and 1-3-5 are overlaid each other with both paths being goals</td><td>
<table>
<tr><td>Method</td><td>DFS</td><td>BFS</td><td>GBFS</td><td>AS</td><td>CUS1</td><td>CUS2</td></tr>
<tr><td>Path</td><td>1-2-4</td><td>1-2-4</td><td>1-2-4</td><td>1-2-4</td><td>1-2-4</td><td>1-2-4</td></tr>
<tr><td>Nodes</td><td>4</td><td>5</td><td>4</td><td>5</td><td>5</td><td>5</td></tr>
</table>
TC09 – Tie Breaking by Chronological Insertion Order: Tests whether nodes inserted earlier are expanded first when values are equal across branches and have the same geological path

All heuristic-based algorithms compute identical h(n) values for both branches, and all cost-based algorithms compute identical g(n) values. Since node 2 has a lower value than 3, it is added to the frontier first and expanded first in all algorithms. DFS and GBFS created only 4 nodes because they reach the goal without needing to expand the second branch at all</td></tr>
<tr><td>10</td><td> 
</td><td>
<table>
<tr><td>Method</td><td>DFS</td><td>BFS</td><td>GBFS</td><td>AS</td><td>CUS1</td><td>CUS2</td></tr>
<tr><td>Path</td><td>1-2-3-4-5</td><td>1-2-3-4-5</td><td>1-2-3-4-5</td><td>1-2-3-4-5</td><td>1-2-3-4-5</td><td>1-2-3-4-5</td></tr>
<tr><td>Nodes</td><td>6</td><td>6</td><td>6</td><td>6</td><td>14</td><td>6</td></tr>
</table>
TC10 – Deep Search Case for DFS: Tests behaviour in a deeper graph where depth-based exploration is required.

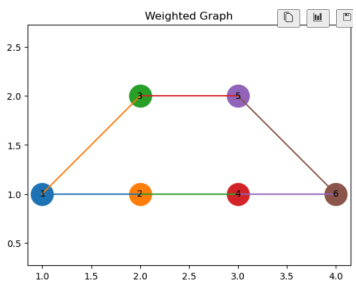
In this test case, CUS1 (IDS) expanded more nodes since the algorithm itself repeatedly explores nodes with DFS with depth limits. Nodes are re-expanded multiple times across iterations which all start from the origin. This repetition increases the amount of expanded nodes.</td></tr>
<tr><td>11</td><td> 
</td><td>
<table>
<tr><td>Method</td><td>DFS</td><td>BFS</td><td>GBFS</td><td>AS</td><td>CUS1</td><td>CUS2</td></tr>
<tr><td>Path</td><td>1-4-7</td><td>1-4-7</td><td>1-4-7</td><td>1-4-7</td><td>1-4-7</td><td>1-4-7</td></tr>
<tr><td>Nodes</td><td>6</td><td>6</td><td>6</td><td>6</td><td>14</td><td>6</td></tr>
</table>
TC11 – Wide Search Case for BFS: Tests behaviour in a wider graph where level-by-level expansion is important.

CUS1 which created 14 nodes due to IDS repeatedly re-expanding nodes from the origin at each depth iteration. Whereas for other algorithms, DFS</td></tr>
</table>

<table>
<tr><td></td><td></td><td>found the goal efficiently since the goal is in the branch where it first explored.</td></tr>
<tr><td>12</td><td> 
</td><td>
<table>
<tr><td>Method</td><td>DFS</td><td>BFS</td><td>GBFS</td><td>AS</td><td>CUS1</td><td>CUS2</td></tr>
<tr><td>Path</td><td>1-2-3-4</td><td>1-2-3-4</td><td>1-2-3-4</td><td>1-2-3-4</td><td>1-2-3-4</td><td>1-2-3-4</td></tr>
<tr><td>Nodes</td><td>4</td><td>4</td><td>4</td><td>4</td><td>7</td><td>4</td></tr>
</table>
TC12 – Cycle in Graph: Tests whether the algorithms avoid infinite loops when cycles exist.

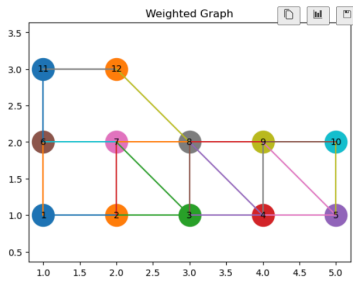
Similar to TC11, IDS re-iterates repeatedly, which causes the high number of expanded nodes.</td></tr>
<tr><td>13</td><td> 

Path cost from 3-5 is 10 whereas the other node path costs are all 1</td><td>
<table>
<tr><td>Method</td><td>DFS</td><td>BFS</td><td>GBFS</td><td>AS</td><td>CUS1</td><td>CUS2</td></tr>
<tr><td>Path</td><td>1-2-4-5</td><td>1-3-5</td><td>1-2-4-5</td><td>1-2-4-5</td><td>1-3-5</td><td>1-2-4-5</td></tr>
<tr><td>Nodes</td><td>5</td><td>5</td><td>5</td><td>5</td><td>7</td><td>5</td></tr>
</table>
TC13 – Different Edge Costs: Tests whether algorithms behave differently when cheapest path and fewest-move path are not the same.

BFS and CUS1 do not take into account path cost so they chose the path that has the least number of nodes. While DFS and GBFS do not take into account path cost, DFS expanded the path with more nodes first and that led it to the goal. As for GBFS it chose 1-2-3-4 over 1-3-5 because node 2 has a lower straight line distance to the goal. That leaves the 2 algorithms AS and CUS2 (Heuristic least move search) considering the path cost, both take the path that is the shortest while also having lower cost.</td></tr>
<tr><td>14</td><td> 

1-2-4-6 and 1-3-5-6 are both valid paths to the goal with same path costs</td><td>
<table>
<tr><td>Method</td><td>DFS</td><td>BFS</td><td>GBFS</td><td>AS</td><td>CUS1</td><td>CUS2</td></tr>
<tr><td>Path</td><td>1-2-4-6</td><td>1-2-4-6</td><td>1-2-4-6</td><td>1-2-4-6</td><td>1-2-4-6</td><td>1-2-4-6</td></tr>
<tr><td>Nodes</td><td>5</td><td>6</td><td>5</td><td>6</td><td>10</td><td>5</td></tr>
</table>
TC14 – Equal Cost Alternative Paths: Tests stable tie-breaking when multiple valid paths have the same total evaluation.

All algorithms chose node 2 instead of the path from node 3 because node 2 is numerically smaller than node 3 and is added to the frontier first. DFS, GBFS and CUS2 created only 5 nodes because they followed the first branch directly to the goal without needing to explore the other branch. BFS and AS created 6 nodes because it expands both node 2 and 3 which have the</td></tr>
</table>

<table>
<tr><td></td><td></td><td>same priority. As mentioned in the above test cases CUS1 performs poorly because it expanded a lot more nodes.</td></tr>
<tr><td>15</td><td> 

All path costs are equal</td><td>
<table>
<tr><td>Method</td><td>DFS</td><td>BFS</td><td>GBFS</td><td>AS</td><td>CUS1</td><td>CUS2</td></tr>
<tr><td>Path</td><td>1-2-3-4-5-10</td><td>1-6-11-12</td><td>1-6-11-12</td><td>1-6-11-12</td><td>1-6-11-12</td><td>1-6-11-12</td></tr>
<tr><td>Nodes</td><td>10</td><td>11</td><td>8</td><td>8</td><td>22</td><td>8</td></tr>
</table>

TC15 – Large Graph Performance Case: Tests whether all algorithms still return valid results on a larger graph within reasonable time.

In this test case we can see that CUS1 (IDS) will have the number of expanded nodes increase alongside the increase in number of nodes and end up having the highest number of expanded nodes. All informed algorithms managed to find the shortest path while having a low number of expanded nodes due to their heuristic guiding them towards the node that is more likely to lead to the goal. DFS fails to find the shortest path and BFS finds the shortest path but expands a higher number of nodes since it checks all paths at each depth</td></tr>
</table>

4. Insights

Implementations	Explanation

Depth-First Search (DFS):
`def DFS(graph, start, goals):`
`  stack = [(start, [start])]`
`  visited = set()`
`  number_of_nodes = 1`
 	
`  while stack:`
`  current_node, path = stack.pop()`
 	
`  if current_node in goals:`
`  return current_node, number_of_nodes, path`
 	
`  if current_node not in visited:`
`  visited.add(current_node)`
 	
`  neighbors = graph.get(current_node, [])`
`  # reverse so the smaller node is expanded first when popped from stack`
`  for neighbor, _ in reversed(neighbors):`
`  if neighbor not in visited:`
`  stack.append((neighbor, path + [neighbor]))`
`  number_of_nodes += 1`
 	
`  return None, number_of_nodes, []`
	• Exploration Strategy: Uses a stack to perform deep searches, prioritizing the exploration of a single branch to its furthest point before backtracking.
	• Cycle Prevention: Employs a visited set to mark explored nodes, ensuring the algorithm stays efficient and avoids infinite loops.
	• Search Tracking: Maintains a number_of_nodes counter that tracks every time a node is added to the stack, measuring the total search effort.

Breadth-FIrst Search (BFS):

```python
from collections import deque


def BFS(graph, start, goals):
    queue = deque([(start, [start])])
    visited = {start}
    number_of_nodes = 1

    while queue:
        current_node, path = queue.popleft()

        if current_node in goals:
            return current_node, number_of_nodes, path

        neighbors = graph.get(current_node, [])
        for neighbor, _ in neighbors:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))
                number_of_nodes += 1

    return None, number_of_nodes, []

```

- Data Structure: Uses a stack (LIFO) to manage the exploration, which forces the algorithm to explore as far as possible along each branch before backtracking.
- Cycle Prevention: Maintains a visited set to keep track of already explored nodes, ensuring the program doesn't get stuck in infinite loops.
- Search Tracking: Increments number_of_nodes every time a neighbor is added to the stack, providing a metric for the total search effort.
- Goal Check: Continually pops the top node from the stack to check if it matches any destination in the goals list.
- Tie-Breaking Logic: The reversed(neighbors) call is crucial; it ensures that the smaller/first neighbor is pushed last and therefore popped first, maintaining a consistent search order.

Greedy Best-First Search (GBFS):

```python
import heapq
from utils import calculate_heuristic


def GBFS(graph, coordinates, start, goals):
    frontier = []
    creation_order = 0
    number_of_nodes = 1
    visited = set()

    heapq.heappush(frontier, (calculate_heuristic(start, goals, coordinates), start, creation_order, [start]))

    while frontier:
        _, current_node, _, path = heapq.heappop(frontier)

        if current_node in goals:
            return current_node, number_of_nodes, path

        if current_node in visited:
            continue

        visited.add(current_node)

        neighbors = graph.get(current_node, [])
        for neighbor, _ in neighbors:
            if neighbor not in visited:
                creation_order += 1
                heuristic = calculate_heuristic(neighbor, goals, coordinates)
                heapq.heappush(frontier, (heuristic, neighbor, creation_order, path + [neighbor]))
                number_of_nodes += 1

    return None, number_of_nodes, []
```

- GBFS is implemented using Python's heapq as a priority queue.
- The frontier stores: heuristic, node, creation order, and path.
- Starts by adding the start node with its heuristic value.
- Uses calculate_heuristic() to estimate distance to the nearest goal.
- Always expands the node with the lowest heuristic first.
- If the current node is a goal, the result is returned immediately.
- Uses a visited set to avoid revisiting nodes.
- Unvisited neighbours are added to the frontier with updated paths.
- creation_order breaks ties when heuristic values are equal.
- number_of_nodes tracks total nodes created.
- Edge costs are ignored, making it faster but not always optimal.

A* Search:

- A* uses Python's heapq as a priority queue.
- Each frontier entry stores: f_cost, node, creation order, current cost, and path.
- Formula: f(n)=g(n)+h(n)
- g(n) = actual accumulated edge cost.
- h(n) = Euclidean distance to the nearest goal.
- Expands the node with the lowest total cost first.
- Uses best_cost instead of a visited set.
- Skips worse paths if a cheaper one already exists.

```python
import heapq
from utils import calculate_heuristic


def AStar(graph, coordinates, start, goals):
    frontier = []
    creation_order = 0
    number_of_nodes = 1

    start_h = calculate_heuristic(start, goals, coordinates)
    heapq.heappush(frontier, (start_h, start, creation_order, 0, [start]))

    best_cost = {start: 0}

    while frontier:
        _, current_node, _, current_cost, path = heapq.heappop(frontier)

        if current_node in goals:
            return current_node, number_of_nodes, path

        if current_cost > best_cost.get(current_node, float('inf')):
            continue

        neighbors = graph.get(current_node, [])
        for neighbor, edge_cost in neighbors:
            new_cost = current_cost + edge_cost

            if new_cost < best_cost.get(neighbor, float('inf')):
                best_cost[neighbor] = new_cost
                creation_order += 1
                heuristic = calculate_heuristic(neighbor, goals, coordinates)
                f_cost = new_cost + heuristic
                heapq.heappush(frontier, (f_cost, neighbor, creation_order, new_cost, path + [neighbor]))
                number_of_nodes += 1

    return None, number_of_nodes, []
```

- Adds neighbours only when a better route is found.
- Makes A* optimal and more reliable than GBFS.

Custom Strategy 1 'Iterative Deepening Search (IDS)':

```python
def _depth_limited_search(graph, current_node, goals, limit, path, visited, counter):
    if current_node in goals:
        return current_node, counter[0], path

    if limit == 0:
        return None, counter[0], []

    visited.add(current_node)

    neighbors = graph.get(current_node, [])
    for neighbor, _ in neighbors:
        if neighbor not in visited:
            counter[0] += 1
            result_goal, result_nodes, result_path = _depth_limited_search(
                graph,
                neighbor,
                goals,
                limit - 1,
                path + [neighbor],
                visited,
                counter,
            )
            if result_goal is not None:
                return result_goal, result_nodes, result_path

    visited.remove(current_node)
    return None, counter[0], []


def CUS1(graph, start, goals):
    # Iterative Deepening Search
    max_depth = len(graph) + 5
    total_nodes_created = 1

    if start in goals:
        return start, total_nodes_created, [start]

    for depth_limit in range(max_depth + 1):
        counter = [total_nodes_created]
        visited = set()
        goal, nodes, path = _depth_limited_search(
            graph, start, goals, depth_limit, [start], visited, counter
        )
        total_nodes_created = nodes
        if goal is not None:
            return goal, total_nodes_created, path

    return None, total_nodes_created, []
```

- IDS is implemented in CUS1() using two parts:
  - _depth_limited_search()
  - CUS1() controller
- _depth_limited_search() performs DFS with a depth limit.
- If the goal is found, it returns the result immediately.
- If the depth limit reaches zero, the search stops for that branch.
- Uses a visited set to avoid cycles.
- Explores unvisited neighbours recursively.
- Updates node count and path during search.
- CUS1() controls the iterative deepening process.
- Starts from depth 0 and increases the limit step by step.
- Calls _depth_limited_search() at each depth.
- Returns when the goal is found, otherwise no solution.
- Combines low memory usage of DFS with the completeness of BFS.

Custom Strategy 2 'Heuristic Least-Moves Search':

- Implemented in the CUS2() function.
- Uses a priority queue (heapq) to expand the most promising node first.
- Each node is scored using moves taken + heuristic estimate to goal.
- Stores best paths in best_moves to avoid worse revisits.
- Expands the lowest-score node each step.
- Returns immediately when the goal is found.
- Better neighbour paths are added back into the frontier.

```python
import heapq
from utils import calculate_heuristic


def CUS2(graph, coordinates, start, goals):
    # Informed search to find shortest path with least moves
    frontier = []
    creation_order = 0
    number_of_nodes = 1

    start_h = calculate_heuristic(start, goals, coordinates)
    heapq.heappush(frontier, (start_h, 0, start, creation_order, [start]))

    best_moves = {start: 0}

    while frontier:
        _, moves, current_node, _, path = heapq.heappop(frontier)

        if current_node in goals:
            return current_node, number_of_nodes, path

        if moves > best_moves.get(current_node, float('inf')):
            continue

        neighbors = graph.get(current_node, [])
        for neighbor, _ in neighbors:
            new_moves = moves + 1

            if new_moves < best_moves.get(neighbor, float('inf')):
                best_moves[neighbor] = new_moves
                creation_order += 1
                heuristic = calculate_heuristic(neighbor, goals, coordinates)
                score = new_moves + heuristic
                heapq.heappush(frontier, (score, new_moves, neighbor, creation_order, path + [neighbor]))
                number_of_nodes += 1

    return None, number_of_nodes, []
```

- Score formula: score=moves+heuristicscore = moves + heuristicscore=moves+heuristic
- Efficient for route-finding problems.

5. Comparison

The searching algorithms have been classified into uninformed (DFS, BFS, IDS) and informed (GBFS, A*, CUS2). The uninformed searching algorithm depends only on graph theory; even though depth-first search has less memory usage and breadth-first search/iterative deepening search guarantees completion of the task, they do not possess scalability. In contrast, informed searching techniques make use of Euclidean distance heuristic to explore the search space. Despite GBFS and Heuristic least moves being faster, they lack optimality.

6. Research

Some additional research was carried out to improve the program and its overall quality. The team studied different search strategies beyond the lecture content in order to select suitable custom algorithms. Based on this research, CUS1 was implemented as Iterative Deepening Search (IDS) which works by repeatedly applying DFS with increasing depth limits. Combines memory efficiency of DFS with completeness of BFS., while CUS2 was implemented as Heuristic Least-Moves Search which uses a heuristic to prioritize nodes that appear to require the fewest moves to reach the goal. It improves efficiency by guiding the search towards promising paths, but does not guarantee an optimal solution (Russell & Norvig, 2021; GeeksforGeeks, 2024). Research was also conducted on heuristic functions that improve informed search performance. Since each node in the graph has coordinates, Euclidean (straight-line) distance was used to estimate how close a node is to the destination. This helped GBFS, A*, and CUS2 algorithms make better decisions during the search process (GeeksforGeeks, 2024). In addition, the team created fifteen different test cases with various graph situations such as cycles, unreachable goals, tie-breaking cases, and larger graphs. These tests were used to compare node creation counts, paths found, and the overall behaviour of each algorithm in various scenarios.

7. Conclusion

While the use of uninformed algorithms may be efficient when applied to straightforward cases, it can consume too much resources when dealing with complicated ones. Fortunately, informed approaches help, and A* algorithm provides the perfect combination of efficiency and accuracy.

8. Acknowledgements/Resources

We would like to thank our Lecturer and Tutor for providing their guidance throughout this assignment.

Credit is given to the AI tools that our group used to aid us in formatting and brainstorming for ideas about this assignment.

Lastly, credits to GeeksforGeeks for information and research regarding the two custom search algorithms

9. References

GeeksforGeeks. (2024). *A* search algorithm. https://www.geeksforgeeks.org/a-search-algorithm/

GeeksforGeeks. (2024). *Iterative deepening search (IDS) in AI*. https://www.geeksforgeeks.org/iterative-deepening-searchids-iterative-deepening-depth-first-searchiddfs/

Siow, P. Z., & Lee, S. C. (2023). Solving the N-puzzle with Iterative Deepening A* (IDA*) algorithm. *Enhanced Knowledge in Sciences and Technology*, *3*(1), 097-107.https://publisher.uthm.edu.my/periodicals/index.php/ekst/article/view/10818/4019

Russell, S. J., & Norvig, P. (2008). 3 Solving Problems by Searching. Pearson Higher Education.